EPIC 07 Task 05 - Complete System Design Interview

Ride Booking System

1. Requirements

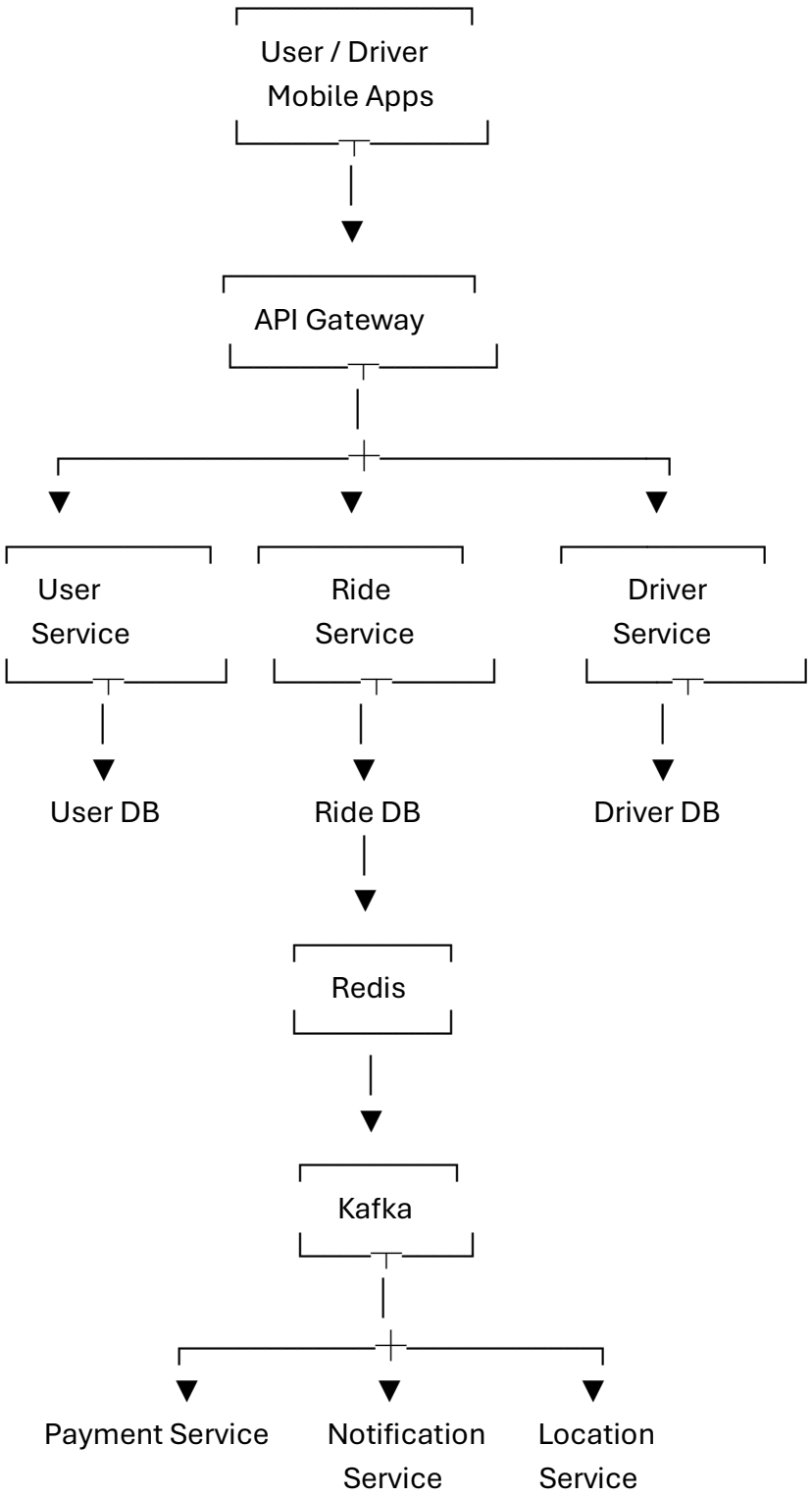
Functional Requirements

- Users can register/login.
- Users can search for available rides.
- Users can book a ride.
- Driver can accept/reject a ride.
- Users can track the driver.
- Users can cancel rides.
- Calculate fare.
- Process payment.
- Send ride and payment notifications.
- Maintain ride history.
- Rate the driver after completing the ride.

Non-Functional Requirements

- High availability.
- Low response time.
- Scalability for millions of users.
- Reliable booking and payment.
- Secure user and payment data.
- Fault tolerance.

2. High-Level Architecture



3. APIs

User APIs

POST /api/users/register
POST /api/users/login
GET /api/users/{id}

Ride APIs

POST /api/rides/request
GET /api/rides/{id}
POST /api/rides/{id}/cancel
POST /api/rides/{id}/complete

Driver APIs

POST /api/drivers/register
PUT /api/drivers/{id}/location
PUT /api/drivers/{id}/availability
POST /api/rides/{id}/accept

Payment APIs

POST /api/payments
GET /api/payments/{id}
POST /api/payments/{id}/refund

4. Database Design

User Table

USER
---
id
name

phone
email
password
created_at

Driver Table

DRIVER
---
id
name
phone
vehicle_number
vehicle_type
status
rating

Ride Table

RIDE
---
id
user_id
driver_id
pickup_location
drop_location
fare
status
created_at
completed_at

Payment Table

PAYMENT
---
id
ride_id
user_id

amount
payment_method
status
transaction_id
created_at

5. Microservices

The system can be divided into these microservices:

User Service
Driver Service
Ride Service
Location Service
Payment Service
Notification Service
Rating Service

Each service has a specific responsibility.

Ride Service

Creates and manages rides.

Driver Service

Manages driver information and availability.

Location Service

Tracks the driver's current location.

Payment Service

Handles payment and refunds.

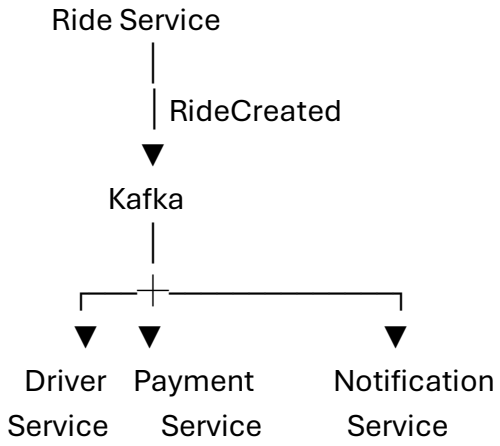
Notification Service

Sends SMS, email, or push notifications.

6. Kafka

Apache Kafka is used for asynchronous communication between services.

Example:



Possible topics:

ride-created
ride-accepted
ride-completed
payment-completed
ride-cancelled

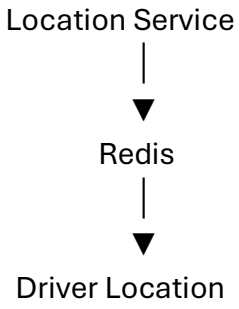
This reduces direct dependency between services.

7. Redis

Redis can be used for fast, frequently changing data.

Examples:

- Driver locations.
- Available drivers.
- User sessions.
- Frequently accessed ride information.

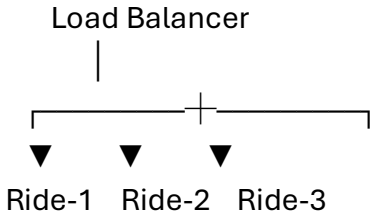


Redis provides very fast reads and helps reduce database load.

8. Scaling

Horizontal Scaling

Multiple instances of each service can run behind a load balancer.



When traffic increases, additional instances can be created.

Database Scaling

- Read replicas for heavy read traffic.
- Database indexing.
- Partitioning/sharding for large datasets.

9. Security

- Use **HTTPS** for communication.
- Use **JWT/OAuth2** for authentication.
- Apply role-based authorization for users and drivers.
- Hash passwords securely.
- Encrypt sensitive information.
- Use secure payment gateways.
- Validate all API inputs.
- Apply API rate limiting.

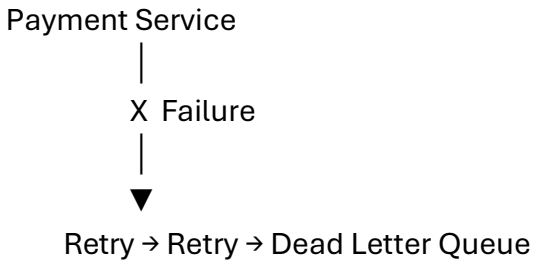
10. Failure Handling

The system should continue operating even when individual services fail.

Techniques

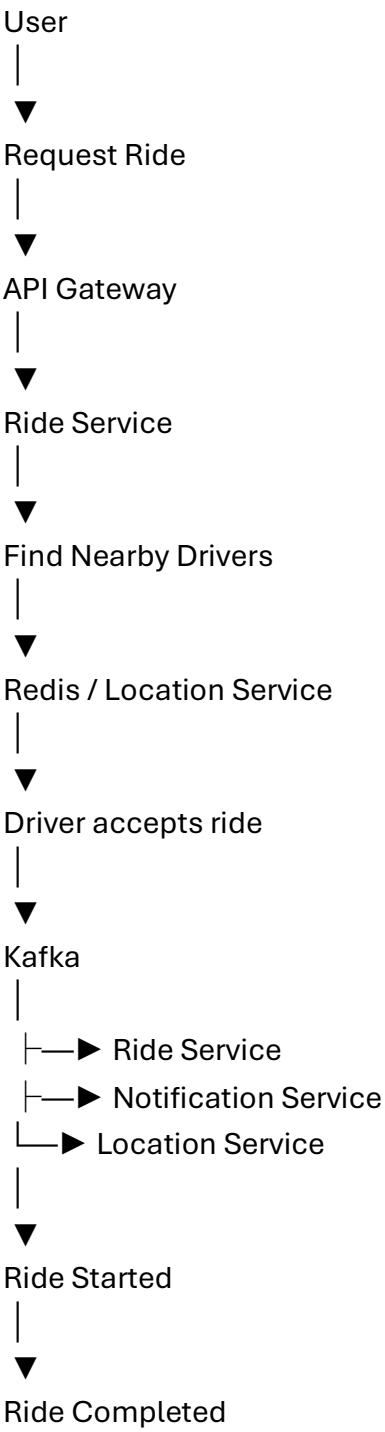
- Retry mechanism.
- Timeout.
- Circuit breaker.
- Kafka messages retry.
- Dead-letter queue.
- Database backups.
- Health checks.
- Multiple service instances.

Example:



The ride should not be marked as paid if the payment service fails.

11. Complete Ride Booking Flow



Final Interview Summary

The **Ride Booking System** uses:

Component	Purpose
API Gateway	Single entry point and routing
Microservices	Separate business functionality
Database	Persistent application data
Redis	Fast caching and driver-location data
Kafka	Asynchronous event communication
Load Balancer	Distributes traffic
Security	Protects users and APIs
Failure Handling	Provides reliability and fault tolerance

This design provides a **scalable, highly available, secure, and reliable ride-booking platform** capable of handling large numbers of users and drivers.